

Fitness Challenge Application

Objective

The objective of this assignment is to build a full-stack Fitness Challenge application designed to gamify physical activity and foster friendly competition among users. The application tracks a diverse set of sports—including running, walking, cycling, gym, swimming, and daily steps—and normalizes their unique metrics into a unified scoring system. This allows users with different exercise preferences to compete fairly on a single global leaderboard while visualizing their personal fitness trends via a dedicated dashboard.

Tech Stack

The application should be developed using the following technologies:

- **Backend:** Any Open-source backend technologies (Java/Node.js/Python etc)
- **Frontend:** Angular/React/Vue frameworks
- **Database:** Relational database (SQLite or an In-Memory DB is recommended)

Core Requirements

1. User Registration API

Create a RESTful POST API to register users with the following requirements:

- Accepts a JSON body for user registration, with the specific fields required left to the candidate's design.
- Returns a unique `userId` upon successful registration.
- The system must prevent duplicate registrations for users with the same first and last names.

2. Data Ingestion & API

Create a RESTful POST API to ingest and serve fitness data supporting specific sports and metrics:

- **Distance (km):** Running, Walking, Cycling
- **Duration (min:sec):** Gym, Swimming
- **Count (integer):** Daily Steps

Candidates may add any additional fields they deem necessary.

The API must validate the request body and return a 400 Bad Request for invalid schemas or mismatched sport/metric types.

3. Scoring System & Normalization

All activities must be normalized into a unified "Points" metric based on the following conversion logic:

•

Activity	Metric	Conversion Rate
Running	1 km ▾	100 Points
Walking	1 km ▾	50 Points
Cycling	1 km ▾	25 Points
Swimming	1 minute ▾	15 Points
Gym	1 minute ▾	5 Points
Daily Steps	100 steps ▾	1 Point

- **Distance:** Points scale linearly based on distance, but the final awarded points cannot be a decimal. Any fractional points must be floored to the nearest integer (e.g., 1.55 km of walking equates to 77.5 points, which floors to 77 points).

- **Duration:** Only fully completed minutes count. Time is floored to the nearest whole minute before point calculation (e.g., 1 minute and 55 seconds counts as 1 minute).
- **Steps:** Points are awarded only for fully completed blocks of 100 steps. Values are floored to the nearest hundred before calculating points (e.g., 399 steps counts as 300 steps, equating to 3 points).

4. Frontend Web Application

Develop a responsive web application featuring:

- Global Leaderboard: Rankings based on total accumulated points and ranking trends.
- Personal Dashboard: Visualizations of activity history, volume over time, and sport preference breakdowns.

Deliverables

Submit the following items for review:

1. **Source Code:** A public link to a Git repository containing the full solution.
2. **Software Design Document (Design Doc / Architecture Spec):** Submit a Design Document (as a PDF, Markdown `DESIGN.md` in the repository, or Google Doc) covering the following sections:

The candidate's Design Document should clearly articulate their engineering decisions and cover the following sections:

- a. **System Architecture & Data Flow:** High-level diagram/overview showing how Frontend, Backend APIs, and Database interact; and a description of request/response flow for registration and activity data ingestion.
- b. **Database Schema & Data Model:** Table/Entity structures (Users, Activities, Leaderboards) with data types and primary/foreign keys; and the enforcement strategy for duplicate user detection.
- c. **API Specifications:** Endpoint routes, payloads, validation rules, error handling, and response structures for User Registration and Data Ingestion APIs.
- d. **Scoring & Normalization Logic:** Implementation details for metric conversions and flooring rules (distance, duration, and 100-step blocks).

- e. **Frontend Architecture & Visualizations:** Component breakdown for the Global Leaderboard and Personal Dashboard, and the strategy for calculating rankings.
- f. **Trade-offs & Edge Cases:** Key technical trade-offs made and edge case handling (e.g., concurrent requests, invalid inputs).

Point of Contact:  Person

Due Date:  Date